МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

Лабораторна робота №2

з курсу: «Технології побудови розподілених баз даних та знань»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Сегментування в Redis

Практичне завдання

1. Встановіть API будь-котрої мови програмування
2. Програмно підключіться щодо будь-котрого шарду в кластері і зчитайте Hash-Slot Map.
3. Програмно зчитайте екземпляри даних з різноманітних шардів.

Хід роботи

1) Підготовка середовища

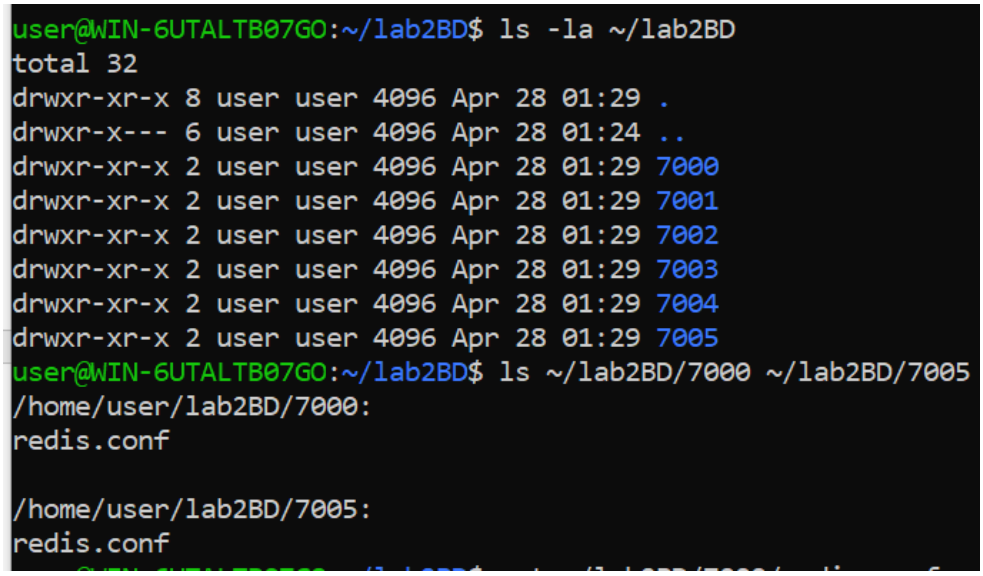
Для виконання роботи підготовлено окремий робочий каталог lab2BD у домашній теці користувача. Перевірено наявність компонентів Redis (redis-server, redis-cli) та інтерпретатора python3, необхідних для подальшого розгортання кластеру та програмної частини завдання.

```
user@WIN-6UTALTB07GO: ~/lab2BD
user@WIN-6UTALTB07GO:~$ mkdir -p ~/lab2BD
user@WIN-6UTALTB07GO:~$ cd ~/lab2BD
user@WIN-6UTALTB07GO:~/lab2BD$ redis-server --version
Redis server v=7.0.15 sha=00000000:0 malloc=jemalloc-5.3.0 bits=64 build=e53ff17674aa6190
user@WIN-6UTALTB07GO:~/lab2BD$ redis-cli --version
redis-cli 7.0.15
user@WIN-6UTALTB07GO:~/lab2BD$ python3 --version
Python 3.12.3
user@WIN-6UTALTB07GO:~/lab2BD$
```

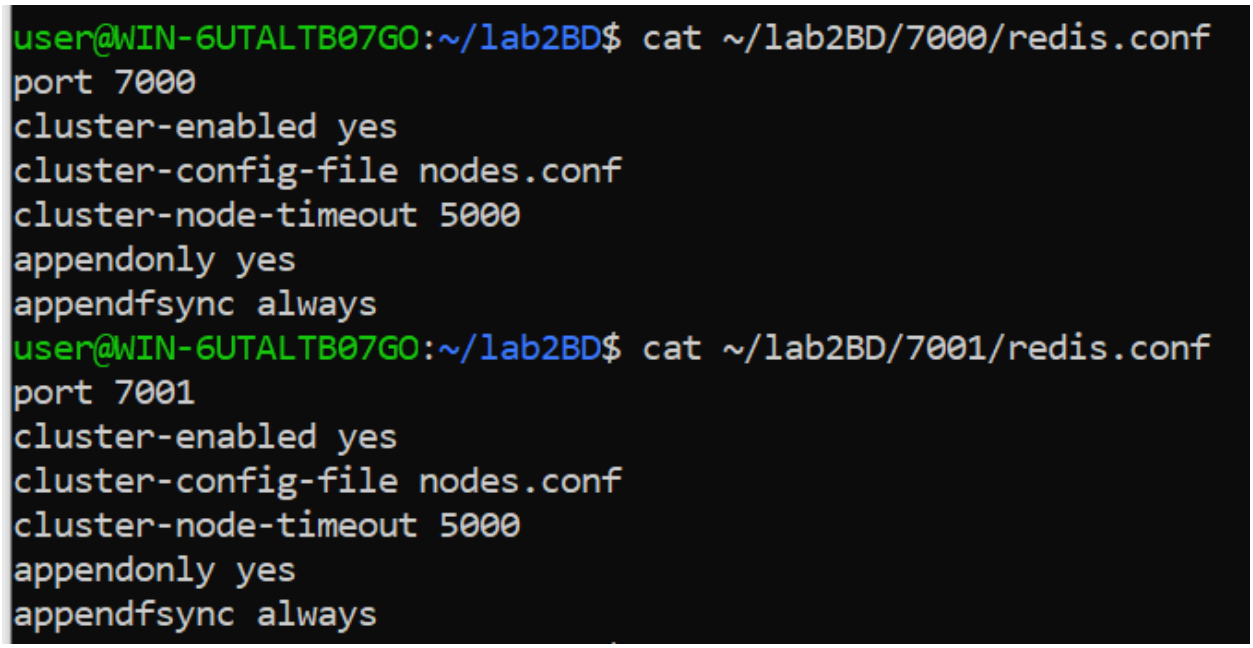
2) Шість вузлів (теки 7000–7005) і файли redis.conf

У робочому каталозі lab2BD створимо шість підкаталогів із іменами, що відповідають номерам портів (7000–7005), щоб кожен майбутній інстанс Redis мав ізольований робочий каталог для службового файлу кластеру nodes.conf та AOF-даних. У кожному підкаталозі сформовано файл redis.conf, у якому задано унікальний номер порту, увімкнено режим кластеру (cluster-enabled yes), вказано ім’я автоматичного конфігу кластеру (cluster-config-file nodes.conf), тайм-аут визнання вузла недоступним (cluster-node-timeout 5000), а також увімкнено журналювання AOF з режимом синхронізації appendfsync always

```
user@WIN-6UTALTB07GO:~/lab2BD$ for port in 7000 7001 7002 7003 7004 7005; do
>   mkdir -p "$port"
>   cat > "$port/redis.conf" <<EOF
> port $port
> cluster-enabled yes
> cluster-config-file nodes.conf
> cluster-node-timeout 5000
> appendonly yes
> appendfsync always
> EOF
> done
```



Всі шість підкаталогів створено з однаковими правами доступу. Перегляд вмісту тек 7000 і 7005 підтверджує наявність у кожному з них файлу redis.conf, що свідчить про успішне завершення скрипта.



Бачимо, що різних теках дійсно зафіксовано різні значення port (7000 і 7001), а решта директив (cluster-enabled, cluster-config-file, cluster-node-timeout, appendonly, appendfsync) ідентична

3) Запуск шести інстансів redis-server

У кожному з підкаталогів 7000–7005 з каталогу lab2BD запустимо окремий процес redis-server з відповідного файлу redis.conf у фоновому режимі, щоб кожен інстанс використовував власний робочий каталог для кластерних і персистентних файлів.

Після невеликої паузи виконаємо перевірку доступності вузлів командами redis-cli -p <порт> PING для портів 7000–7005;

```
user@WIN-6UTALTB07GO:~/lab2BD$ for port in 7000 7001 7002 7003 7004 7005; do
>   (cd "$port" && redis-server ./redis.conf &)
> done

sleep 2

for port in 7000 7001 7002 7003 7004 7005; do
  redis-cli -p "$port" PING
```

```
doneuser@WIN-6UTALTB07GO:~/lab2BD$
512:C 28 Apr 2026 01:56:09.952 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
512:C 28 Apr 2026 01:56:09.952 # Redis version=7.0.15, bits=64, commit=00000000, modified=0, pid=512, just started
512:C 28 Apr 2026 01:56:09.952 # Configuration loaded
user@WIN-6UTALTB07GO:~/lab2BD$ sleep 2
519:C 28 Apr 2026 01:56:09.954 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
519:C 28 Apr 2026 01:56:09.954 # Redis version=7.0.15, bits=64, commit=00000000, modified=0, pid=519, just started
519:C 28 Apr 2026 01:56:09.954 # Configuration loaded
512:M 28 Apr 2026 01:56:09.955 * monotonic clock: POSIX clock_gettime
519:M 28 Apr 2026 01:56:09.955 * monotonic clock: POSIX clock_gettime
512:M 28 Apr 2026 01:56:09.958 * No cluster configuration found, I'm 557f0eb61607fd93ff2f13aefc7cce829f60f44d
516:C 28 Apr 2026 01:56:09.959 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
516:C 28 Apr 2026 01:56:09.959 # Redis version=7.0.15, bits=64, commit=00000000, modified=0, pid=516, just started
516:C 28 Apr 2026 01:56:09.959 # Configuration loaded
516:M 28 Apr 2026 01:56:09.959 * monotonic clock: POSIX clock_gettime
519:M 28 Apr 2026 01:56:09.961 * No cluster configuration found, I'm ed01216f9160866f6aad541a8bc6a16bbd7fc154
522:C 28 Apr 2026 01:56:09.961 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
522:C 28 Apr 2026 01:56:09.961 # Redis version=7.0.15, bits=64, commit=00000000, modified=0, pid=522, just started
522:C 28 Apr 2026 01:56:09.961 # Configuration loaded
522:M 28 Apr 2026 01:56:09.961 * monotonic clock: POSIX clock_gettime
522:M 28 Apr 2026 01:56:09.962 * No cluster configuration found, I'm 91f6168c56e4d45cf845b23377a2f8bbcb8f857d
516:M 28 Apr 2026 01:56:09.962 * No cluster configuration found, I'm ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
527:C 28 Apr 2026 01:56:09.963 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
527:C 28 Apr 2026 01:56:09.963 # Redis version=7.0.15, bits=64, commit=00000000, modified=0, pid=527, just started
527:C 28 Apr 2026 01:56:09.963 # Configuration loaded
527:M 28 Apr 2026 01:56:09.963 * monotonic clock: POSIX clock_gettime
527:M 28 Apr 2026 01:56:09.965 * No cluster configuration found, I'm b669df19ffb912b64892b52b5fa6edc1854979a4
524:C 28 Apr 2026 01:56:09.966 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
524:C 28 Apr 2026 01:56:09.966 # Redis version=7.0.15, bits=64, commit=00000000, modified=0, pid=524, just started
524:C 28 Apr 2026 01:56:09.966 # Configuration loaded
524:M 28 Apr 2026 01:56:09.966 * monotonic clock: POSIX clock_gettime
524:M 28 Apr 2026 01:56:09.967 * No cluster configuration found, I'm a1d050d1557b91a766f3a8de1c87335579761a1a
```

за отриманих відповідей PONG усіх шість процесів вважалося успішно запущеним і готовим до подальшого об’єднання в кластер за допомогою утиліти redis-cli.

```
user@WIN-6UTALTB07GO:~/lab2BD$ for port in 7000 7001 7002 7003 7004 7005; do
>   redis-cli -p "$port" PING
> done
PONG
PONG
PONG
PONG
PONG
PONG
```

4)Створення кластеру: три master і три replica (--cluster-replicas 1)

Командою redis-cli --cluster create з переліком адрес 127.0.0.1:7000–127.0.0.1:7005 та параметром --cluster-replicas 1 було ініційоване створення кластеру з шести вузлів.

```
user@WIN-6UTALTB07GO: ~
user@WIN-6UTALTB07GO:~$ redis-cli --cluster create \
>   127.0.0.1:7000 127.0.0.1:7001 127.0.0.1:7002 \
>   127.0.0.1:7003 127.0.0.1:7004 127.0.0.1:7005 \
-cluster>   --cluster-replicas 1
```

Після виконання команди утиліта розподілила 16384 hash-слоти між трьома вузлами типу master: діапазони 0–5460, 5461–10922 та 10923–16383. Три інші вузли призначено репліками відповідно до автоматично сформованого плану: репліка на порту 7004 відповідає master на 7000, репліка 7005 — master на 7001, репліка 7003 — master на 7002. Таким чином, задано топологію «три основні вузли та по одній дублювальній копії на кожен основний». З’явлене попередження про розміщення деяких реплік на тому ж хості, що й master, є очікуваним для локальної конфігурації на одній машині й не означає помилку виконання. Після підтвердження конфігурації відповіді yes кластер ініційовано командою CLUSTER MEET, виконано перевірку узгодженості конфігурації слотів.

```
>>> Performing hash slots allocation on 6 nodes...
Master[0] -> Slots 0 - 5460
Master[1] -> Slots 5461 - 10922
Master[2] -> Slots 10923 - 16383
Adding replica 127.0.0.1:7004 to 127.0.0.1:7000
Adding replica 127.0.0.1:7005 to 127.0.0.1:7001
Adding replica 127.0.0.1:7003 to 127.0.0.1:7002
```

```
>>> Trying to optimize slaves allocation for anti-affinity
[WARNING] Some slaves are in the same host as their master
M: 557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000
   slots:[0-5460] (5461 slots) master
M: ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001
   slots:[5461-10922] (5462 slots) master
M: ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002
   slots:[10923-16383] (5461 slots) master
S: 91f6168c56e4d45cf845b23377a2f8bbcb8f857d 127.0.0.1:7003
   replicates ed01216f9160866f6aad541a8bc6a16bbd7fc154
S: a1d050d1557b91a766f3a8de1c87335579761a1a 127.0.0.1:7004
   replicates 557f0eb61607fd93ff2f13aefc7cce829f60f44d
S: b669df19ffb912b64892b52b5fa6edc1854979a4 127.0.0.1:7005
   replicates ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
Can I set the above configuration? (type 'yes' to accept): yes
>>> Nodes configuration updated
>>> Assign a different config epoch to each node
>>> Sending CLUSTER MEET messages to join the cluster
Waiting for the cluster to join

>>> Performing Cluster Check (using node 127.0.0.1:7000)
M: 557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000
   slots:[0-5460] (5461 slots) master
   1 additional replica(s)
S: a1d050d1557b91a766f3a8de1c87335579761a1a 127.0.0.1:7004
   slots: (0 slots) slave
   replicates 557f0eb61607fd93ff2f13aefc7cce829f60f44d
M: ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001
   slots:[5461-10922] (5462 slots) master
   1 additional replica(s)
S: 91f6168c56e4d45cf845b23377a2f8bbcb8f857d 127.0.0.1:7003
   slots: (0 slots) slave
   replicates ed01216f9160866f6aad541a8bc6a16bbd7fc154
M: ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002
   slots:[10923-16383] (5461 slots) master
   1 additional replica(s)
S: b669df19ffb912b64892b52b5fa6edc1854979a4 127.0.0.1:7005
   slots: (0 slots) slave
   replicates ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
user@WIN-6UTALTB07GO:~$
```

У результаті отримано повідомлення про успішне покриття слотів: [OK] All 16384 slots covered, що підтверджує повну адресацію всіх hash-слотів і статусність кластеру до подальших операцій.

5)Масштабування: вузли 7006 (master) і 7007 (replica), add-node і reshard

```
user@WIN-6UTALTB07GO:~/lab2BD$ for port in 7006 7007; do
>   mkdir -p "$port"
>   cat > "$port/redis.conf" <<EOF
> port $port
> cluster-enabled yes
> cluster-config-file nodes.conf
> cluster-node-timeout 5000
> appendonly yes
> appendfsync always
> EOF
> done
```

```
2066:M 28 Apr 2026 20:04:07.263 * Creating AOF base file appendonly.aof.1.base.rdb on server start
2063:M 28 Apr 2026 20:04:07.263 * Creating AOF base file appendonly.aof.1.base.rdb on server start
2063:M 28 Apr 2026 20:04:07.271 * Creating AOF incr file appendonly.aof.1.incr.aof on server start
2066:M 28 Apr 2026 20:04:07.271 * Creating AOF incr file appendonly.aof.1.incr.aof on server start
2063:M 28 Apr 2026 20:04:07.271 * Ready to accept connections
2066:M 28 Apr 2026 20:04:07.271 * Ready to accept connections
user@WIN-6UTALTB07GO:~/lab2BD$ redis-cli -p 7006 PING
PONG
user@WIN-6UTALTB07GO:~/lab2BD$ redis-cli -p 7007 PING
PONG
user@WIN-6UTALTB07GO:~/lab2BD$
```

Додамо новий master: 7006, підключитися до живого кластеру через вже існуючий вузол redis-cli --cluster add-node 127.0.0.1:7006 127.0.0.1:7000

```
user@WIN-6UTALTB07GO: ~
user@WIN-6UTALTB07GO:~$ redis-cli --cluster add-node 127.0.0.1:7006 127.0.0.1:7000
>>> Adding node 127.0.0.1:7006 to cluster 127.0.0.1:7000
>>> Performing Cluster Check (using node 127.0.0.1:7000)
M: 557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000
   slots:[0-5460] (5461 slots) master
   1 additional replica(s)
S: a1d050d1557b91a766f3a8de1c87335579761a1a 127.0.0.1:7004
   slots: (0 slots) slave
   replicates 557f0eb61607fd93ff2f13aefc7cce829f60f44d
M: ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001
   slots:[5461-10922] (5462 slots) master
   1 additional replica(s)
S: 91f6168c56e4d45cf845b23377a2f8bbcb8f857d 127.0.0.1:7003
   slots: (0 slots) slave
   replicates ed01216f9160866f6aad541a8bc6a16bbd7fc154
M: ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002
   slots:[10923-16383] (5461 slots) master
   1 additional replica(s)
S: b669df19ffb912b64892b52b5fa6edc1854979a4 127.0.0.1:7005
   slots: (0 slots) slave
   replicates ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
>>> Getting functions from cluster
>>> Send FUNCTION LIST to 127.0.0.1:7006 to verify there is no functions in it
>>> Send FUNCTION RESTORE to 127.0.0.1:7006
>>> Send CLUSTER MEET to node 127.0.0.1:7006 to make it join the cluster.
```

Зчитаємо ноди

```
user@WIN-6UTALTB07GO:~$ redis-cli -p 7000 cluster nodes
557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000@17000 myself,master - 0 1777395953000 1 connected 0-5460
a1d050d1557b91a766f3a8de1c87335579761a1a 127.0.0.1:7004@17004 slave 557f0eb61607fd93ff2f13aefc7cce829f60f44d 0 1777395952503 1 connected
ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001@17001 master - 0 1777395953000 2 connected 5461-10922
91f6168c56e4d45cf845b23377a2f8bbcb8f857d 127.0.0.1:7003@17003 slave ed01216f9160866f6aad541a8bc6a16bbd7fc154 0 1777395952503 3 connected
ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002@17002 master - 0 1777395953916 3 connected 10923-16383
ac8bc4722692a7875a28d0076ce044d2e87be60a 127.0.0.1:7006@17006 master - 0 1777395953000 0 connected
b669df19ffb912b64892b52b5fa6edc1854979a4 127.0.0.1:7005@17005 slave ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 0 1777395953512 2 connected
```

Додамо replica: 7007 → репліка master 7006, вказуємо, що новий вузол — slave і його master — за ID.

```
user@WIN-6UTALTB07GO:~/lab2BD/7007$ export MASTER_ID_7006="ac8bc4722692a7875a28d0076ce044d2e87be60a"
user@WIN-6UTALTB07GO:~/lab2BD/7007$
user@WIN-6UTALTB07GO:~/lab2BD/7007$ redis-cli --cluster add-node 127.0.0.1:7007 127.0.0.1:7000 \
>   --cluster-slave --cluster-master-id "$MASTER_ID_7006"
>>> Adding node 127.0.0.1:7007 to cluster 127.0.0.1:7000
>>> Performing Cluster Check (using node 127.0.0.1:7000)
M: 557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000
   slots:[0-5460] (5461 slots) master
   1 additional replica(s)
S: a1d050d1557b91a766f3a8de1c87335579761a1a 127.0.0.1:7004
   slots: (0 slots) slave
   replicates 557f0eb61607fd93ff2f13aefc7cce829f60f44d
M: ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001
   slots:[5461-10922] (5462 slots) master
   1 additional replica(s)
S: fa966a765e12f70bcd8f7614cc6d8b6f66856f09 127.0.0.1:7007
   slots: (0 slots) slave
   replicates ac8bc4722692a7875a28d0076ce044d2e87be60a
S: 91f6168c56e4d45cf845b23377a2f8bbcb8f857d 127.0.0.1:7003
   slots: (0 slots) slave
   replicates ed01216f9160866f6aad541a8bc6a16bbd7fc154
M: ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002
   slots:[10923-16383] (5461 slots) master
   1 additional replica(s)
M: ac8bc4722692a7875a28d0076ce044d2e87be60a 127.0.0.1:7006
   slots: (0 slots) master
   1 additional replica(s)
S: b669df19ffb912b64892b52b5fa6edc1854979a4 127.0.0.1:7005
   slots: (0 slots) slave
   replicates ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
```

Виконаємо reshard (передамо 4096 слотів на master 7006)

```
user@WIN-6UTALTB07GO:~/lab2BD/7007$ redis-cli --cluster reshard 127.0.0.1:7000
>>> Performing Cluster Check (using node 127.0.0.1:7000)
M: 557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000
   slots:[0-5460] (5461 slots) master
   1 additional replica(s)
S: a1d050d1557b91a766f3a8de1c87335579761a1a 127.0.0.1:7004
   slots: (0 slots) slave
   replicates 557f0eb61607fd93ff2f13aefc7cce829f60f44d
M: ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001
   slots:[5461-10922] (5462 slots) master
   1 additional replica(s)
S: fa966a765e12f70bcd8f7614cc6d8b6f66856f09 127.0.0.1:7007
   slots: (0 slots) slave
   replicates ac8bc4722692a7875a28d0076ce044d2e87be60a
S: 91f6168c56e4d45cf845b23377a2f8bbcb8f857d 127.0.0.1:7003
   slots: (0 slots) slave
   replicates ed01216f9160866f6aad541a8bc6a16bbd7fc154
M: ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002
   slots:[10923-16383] (5461 slots) master
   1 additional replica(s)
M: ac8bc4722692a7875a28d0076ce044d2e87be60a 127.0.0.1:7006
   slots: (0 slots) master
   1 additional replica(s)
S: b669df19ffb912b64892b52b5fa6edc1854979a4 127.0.0.1:7005
   slots: (0 slots) slave
   replicates ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
```

```
Ready to move 4096 slots.
  Source nodes:
    M: 557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000
       slots:[0-5460] (5461 slots) master
       1 additional replica(s)
    M: ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001
       slots:[5461-10922] (5462 slots) master
       1 additional replica(s)
    M: ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002
       slots:[10923-16383] (5461 slots) master
       1 additional replica(s)
  Destination node:
    M: ac8bc4722692a7875a28d0076ce044d2e87be60a 127.0.0.1:7006
       slots: (0 slots) master
       1 additional replica(s)
  Resharding plan:
    Moving slot 5461 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5462 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5463 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5464 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5465 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5466 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5467 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5468 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5469 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5470 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5471 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5472 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
    Moving slot 5473 from ed3e967ae49c7f0b895b62d7ef2b98633d2722c5
```

Перевіримо слоти та ноди:

```
user@WIN-6UTALTB07GO:~/lab2BD/7007$ redis-cli -p 7000 cluster slots
1) 1) (integer) 0
   2) (integer) 1364
   3) 1) "127.0.0.1"
      2) (integer) 7006
      3) "ac8bc4722692a7875a28d0076ce044d2e87be60a"
      4) (empty array)
   4) 1) "127.0.0.1"
      2) (integer) 7007
      3) "fa966a765e12f70bcd8f7614cc6d8b6f66856f09"
      4) (empty array)
2) 1) (integer) 1365
   2) (integer) 5460
   3) 1) "127.0.0.1"
      2) (integer) 7000
      3) "557f0eb61607fd93ff2f13aefc7cce829f60f44d"
      4) (empty array)
   4) 1) "127.0.0.1"
      2) (integer) 7004
      3) "a1d050d1557b91a766f3a8de1c87335579761a1a"
      4) (empty array)
3) 1) (integer) 5461
   2) (integer) 6826
   3) 1) "127.0.0.1"
      2) (integer) 7006
      3) "ac8bc4722692a7875a28d0076ce044d2e87be60a"
      4) (empty array)
   4) 1) "127.0.0.1"
      2) (integer) 7007
      3) "fa966a765e12f70bcd8f7614cc6d8b6f66856f09"
      4) (empty array)
4) 1) (integer) 6827
   2) (integer) 10922
   3) 1) "127.0.0.1"
      2) (integer) 7001
      3) "ed3e967ae49c7f0b895b62d7ef2b98633d2722c5"
      4) (empty array)
   4) 1) "127.0.0.1"
      2) (integer) 7005
      3) "b669df19ffb912b64892b52b5fa6edc1854979a4"
      4) (empty array)
5) 1) (integer) 10923
   2) (integer) 12287
   3) 1) "127.0.0.1"
      2) (integer) 7006
      3) "ac8bc4722692a7875a28d0076ce044d2e87be60a"
      4) (empty array)
   4) 1) "127.0.0.1"
      2) (integer) 7007
      3) "fa966a765e12f70bcd8f7614cc6d8b6f66856f09"
      4) (empty array)
6) 1) (integer) 12288
   2) (integer) 16383
   3) 1) "127.0.0.1"
      2) (integer) 7002
      3) "ed01216f9160866f6aad541a8bc6a16bbd7fc154"
      4) (empty array)
   4) 1) "127.0.0.1"
      2) (integer) 7003
      3) "91f6168c56e4d45cf845b23377a2f8bbcb8f857d"
      4) (empty array)
```

```
    4) (empty array)
user@WIN-6UTALTB07GO:~/lab2BD/7007$ redis-cli -p 7000 cluster nodes
557f0eb61607fd93ff2f13aefc7cce829f60f44d 127.0.0.1:7000@17000 myself,master - 0 1777406121000 1 connected 1365-5460
a1d050d1557b91a766f3a8de1c87335579761a1a 127.0.0.1:7004@17004 slave 557f0eb61607fd93ff2f13aefc7cce829f60f44d 0 1777406122299 1 connected
ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 127.0.0.1:7001@17001 master - 0 1777406121292 2 connected 6827-10922
fa966a765e12f70bcd8f7614cc6d8b6f66856f09 127.0.0.1:7007@17007 slave ac8bc4722692a7875a28d0076ce044d2e87be60a 0 1777406122601 7 connected
91f6168c56e4d45cf845b23377a2f8bbcb8f857d 127.0.0.1:7003@17003 slave ed01216f9160866f6aad541a8bc6a16bbd7fc154 0 1777406122000 3 connected
ed01216f9160866f6aad541a8bc6a16bbd7fc154 127.0.0.1:7002@17002 master - 0 1777406122000 3 connected 12288-16383
ac8bc4722692a7875a28d0076ce044d2e87be60a 127.0.0.1:7006@17006 master - 0 1777406122601 7 connected 0-1364 5461-6826 10923-12287
b669df19ffb912b64892b52b5fa6edc1854979a4 127.0.0.1:7005@17005 slave ed3e967ae49c7f0b895b62d7ef2b98633d2722c5 0 1777406120583 2 connected
```

6) Встановлення API

user@WIN-6UTALTB07GO: ~/lab2BD

```
user@WIN-6UTALTB07GO:~$ cd ~/lab2BD
user@WIN-6UTALTB07GO:~/lab2BD$ python3 -m venv venv
user@WIN-6UTALTB07GO:~/lab2BD$ source venv/bin/activate
(venv) user@WIN-6UTALTB07GO:~/lab2BD$ pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.1-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 4.3 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.1
(venv) user@WIN-6UTALTB07GO:~/lab2BD$ pip install "redis>=4.2.0"
Collecting redis>=4.2.0
  Using cached redis-7.4.0-py3-none-any.whl.metadata (12 kB)
Using cached redis-7.4.0-py3-none-any.whl (409 kB)
Installing collected packages: redis
Successfully installed redis-7.4.0
(venv) user@WIN-6UTALTB07GO:~/lab2BD$ pip show redis
Name: redis
Version: 7.4.0
Summary: Python client for Redis database and key-value store
Home-page: https://github.com/redis/redis-py
Author:
Author-email: "Redis Inc." <oss@redis.com>
License-Expression: MIT
Location: /home/user/lab2BD/venv/lib/python3.12/site-packages
Requires:
Required-by:
(venv) user@WIN-6UTALTB07GO:~/lab2BD$ cd ~/lab2BD
(venv) user@WIN-6UTALTB07GO:~/lab2BD$ source venv/bin/activate
(venv) user@WIN-6UTALTB07GO:~/lab2BD$ nano task.py
(venv) user@WIN-6UTALTB07GO:~/lab2BD$
```

7) Підключення до кластеру і зчитування Hash – Slot Map та зчитування даних, розміщених на різних шардах

```
(venv) user@WIN-6UTALTB07GO:~/lab2BD$ python3 task.py
=== CLUSTER SLOTS (Hash-Slot Map) ===
{(0, 1364): {'primary': ('127.0.0.1', 7006), 'replicas': [('127.0.0.1', 7007)]},
 (1365, 5460): {'primary': ('127.0.0.1', 7000),
                'replicas': [('127.0.0.1', 7004)]},
 (5461, 6826): {'primary': ('127.0.0.1', 7006),
                'replicas': [('127.0.0.1', 7007)]},
 (6827, 10922): {'primary': ('127.0.0.1', 7001),
                 'replicas': [('127.0.0.1', 7005)]},
 (10923, 12287): {'primary': ('127.0.0.1', 7006),
                  'replicas': [('127.0.0.1', 7007)]},
 (12288, 16383): {'primary': ('127.0.0.1', 7002),
                  'replicas': [('127.0.0.1', 7003)]}}

=== SET / GET (KEYSLOT + значення) ===
'foo': KEYSLOT -> 12182
'name': KEYSLOT -> 5798
'project': KEYSLOT -> 5904

GET 'foo' -> alice
GET 'name' -> bob
GET 'project' -> lab2-bd
```

Висновок.

У ході виконання лабораторної роботи розглянуто та практично реалізовано механізм сегментації даних у Redis для кластерної топології на основі хешових слотів. Навчальний кластер зібрано на локальному вузлі через кілька ізольованих каталогів із власними redis.conf, увімкнено кластерний режим і журналювання AOF, виконано формування першого кластеру з шести вузлів і чергою одиниць масштабування (--cluster-replicas 1) із повним покриттям 16384 слотів. Закріплено розуміння ролей вузлів типу master і replica та узгодженості між слотами та відповідальністю основних процесів за діапазонами ключів.

Продемонстровано сценарій розширення кластера введенням додаткових вузлів на портах 7006 і 7007, їхнього приєднання утилітою redis-cli --cluster add-node, призначення репліки за ідентифікатором master і виконання перерозподілу слотів (reshard) з переміщенням 4096 слотів частково із наявних master на новий основний вузол. Отримані стани топології зафіксовано через CLUSTER NODES та CLUSTER SLOTS, що підтверджує узгодженість розподілу слотів після операцій масштабування.

Завершальний практичний етап реалізовано мовою Python за допомогою клієнтської бібліотеки redis і класу кластерного з’єднання: із

підключенням до єдиної точки входу здійснено програмне отримання мапи діапазонів слотів (відповідник CLUSTER SLOTS), а також запис і читання кількох ключів із обчисленням номерів hash-слотів. Таким чином, підтверджено здатність програмного клієнта прозоро маршрутизувати звернення між вузлами кластеру відповідно до ключів і конфігурації слотів, що відповідає умовам практичного завдання та пояснює роботу кластера Redis із точки зору прикладного застосунку та адміністрування.

Запитання для закріплення знань

1. Переваги й вади слотів при шардингу

Redis кластер поділяє ключовий простір на 16384 hash слоти, а не на довільні групи ключів напряму. Кожному діапазону слотів відповідає конкретний майстер, репліками покривають відказостійкість.

Переваги: перенаправлення навантаження роблять порціями через діапазони слотів, без переносів «ключ за ключем». Відповідність ключ → сліт задається CRC16 по імені ключа, тому локальність передбачувана без глобальної таблиці всіх ключів, достатньо актуального списку діапазон слотів у кластері.

Вади й ризики: будь який перерозподіл слотів потребує узгодженої мапи навколо кластеру; якщо клієнт має застарілі дані, з’являються MOVED, простий клієнт може давати зайві переходи або помилки. reshard це складна операція потенційної тимчасової неузгодженості до завершення. Нового майстра можна підняти без слотів, якщо не зробити перерозподіл: тоді він «порожній» для даних. Тож слоти дають керованість, але вимагають дисципліни і кластерно свідомих клієнтів.

2. Які додаткові запити щодо кластера реалізовані в API

Розрізняють команди Redis (протокол) і обгортки в клієнті (наприклад Python redis). У протоколі: CLUSTER INFO, CLUSTER NODES, CLUSTER SLOTS, CLUSTER KEYSLOT (номер слота для рядка ключа), а також логіка MOVED / ASK для маршрутизації та перехідних станів. У бібліотеці: зазвичай є RedisCluster (вхід з одного вузла, оновлення мапи слотів) і прямий виклик execute_command("CLUSTER", ...) для сирих відповідей. У новіших версіях можуть бути зручні методи на кшталт cluster_slots / cluster_nodes, по суті це ті ж команди.

redis-cli --cluster ... (create, add-node, reshard, check) це інструмент адміністратора, не частина Python API, але в звіті його логічно зіставляють з тим, що програма читає через CLUSTER SLOTS.

Додаток А

```python
from pprint import pprint

from redis.cluster import ClusterNode, RedisCluster

STARTUP_NODES = [
    ClusterNode(host="127.0.0.1", port=7000),
]


def main() -> None:
    rc = RedisCluster(
        startup_nodes=STARTUP_NODES,
        decode_responses=True,
    )

    # 1) Hash-Slot map: те саме, що redis-cli CLUSTER SLOTS
    slots_raw = rc.execute_command("CLUSTER SLOTS")

    print("=== CLUSTER SLOTS (Hash-Slot Map) ===")
    pprint(slots_raw)
    print()

    # 2) Запис і читання ключів (різні слоти → різні шарди; клієнт ходить
сам)
    keys_values = [
        ("foo", "alice"),
        ("name", "bob"),
        ("project", "lab2-bd"),
    ]
    print("=== SET / GET (KEYSLOT + значення) ===")
    for key, val in keys_values:
        rc.set(key, val)
        slot = rc.cluster_keyslot(key)
        print(f"{key!r}: KEYSLOT -> {slot}")

    print()
    for key, _ in keys_values:
        print(f"GET {key!r} -> {rc.get(key)}")

    rc.close()


if __name__ == "__main__":
    main()
```